

Tarefa 6: Estimativa de π com Monte Carlo e OpenMP

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Junho de 2026

1. Introdução

Este relatório documenta a implementação e análise da estimativa estocástica do número π pelo método de Monte Carlo, paralelizada com OpenMP. O experimento explora progressivamente os problemas de condição de corrida, as formas de corrigi-los e o impacto de diferentes estratégias de sincronização no desempenho.

2. Algoritmo Monte Carlo

O método consiste em gerar N pontos aleatórios (x, y) no quadrado unitário $[0,1)^2$ e verificar quantos caem dentro do quarto de círculo de raio 1. A razão entre pontos dentro e o total é $\pi/4$:

$$\pi \approx 4 \times (\text{pontos dentro do círculo}) / N$$

2.1 Ambiente de Execução

Parâmetro	Valor
CPU	AMD Ryzen 5 5600G
Núcleos físicos	4 (8 threads com SMT)
Compilador	GCC com -O2 -fopenmp
N (pontos)	100.000.000
Threads	4

3. Versões Implementadas

3.1 Versão Sequencial

Implementação de referência. Utiliza `rand_r()` — função reentrante que recebe a seed por ponteiro — garantindo reprodutibilidade. Não há paralelismo; serve como baseline para cálculo de speedup.

3.2 Versão Paralela Ingênua (Race Condition)

Adiciona `#pragma omp parallel for` sem qualquer proteção. Apresenta dois problemas simultâneos:

a) `dentro++` não é atômica (`LOAD` → `ADD` → `STORE`); múltiplas threads intercalam as operações e incrementos são perdidos.

b) `seed` é compartilhada: threads concorrem no próprio RNG, corrompendo a geração de números aleatórios.

Resultado: valor de π varia a cada execução e é incorreto — comportamento típico de data race.

3.3 Com `#pragma omp critical`

Envolve o incremento em uma seção crítica, tornando a versão correta. Porém, o lock global é adquirido uma vez por ponto que cai dentro do círculo — operação proporcional a N . Isso serializa praticamente toda a região paralela, tornando esta versão mais lenta que a sequencial.

3.4 Com `parallel + for + Cláusulas`

Versão didática que demonstra o uso explícito das cláusulas de escopo do OpenMP. Cada thread acumula sua contagem parcial em uma variável local e executa a seção crítica apenas uma vez, ao final — custo fixo independente de N .

Cláusula	Comportamento
shared	Variável visível e compartilhada por todas as threads. Acesso a dentro protegido por critical.
firstprivate	Cada thread recebe cópia inicializada com o valor do master (SEED). Permite derivar seed única via XOR com thread ID.
private	Cópia privada sem valor inicial pelo OpenMP; programador inicializa explicitamente (<code>local = 0</code>).
lastprivate	Ao fim do for, a variável recebe o valor do thread que executou a última iteração lógica ($i = N-1$).
default(none)	Exige declaração explícita de cada variável; o compilador rejeita omissões — força análise de escopo.

3.5 Com `reduction` (idiomático)

O OpenMP cria automaticamente uma cópia privada de dentro por thread, acumula em paralelo e soma os parciais ao final. Não exige `critical` explícito — é a forma mais eficiente e recomendada para somas paralelas.

4. Resultados

Versão	π estimado	Tempo (s)	Correto?
Sequencial	3,14153268	0,94	Sim
Ingênua — execução 1	2,42180700	1,73	NÃO
Ingênua — execução 2	2,56800936	1,91	NÃO
Ingênua — execução 3	1,38720932	2,77	NÃO
Com omp critical	3,14158028	6,10	Sim
parallel+for+cláusulas	3,14139304	0,25	Sim
Com reduction	3,14159384	0,21	Sim

Nota: As três execuções da versão ingênua foram realizadas consecutivamente com o mesmo binário. Os valores distintos de π a cada execução confirmam o comportamento não-determinístico de condições de corrida.

5. Análise e Discussão

Os resultados evidenciam três padrões distintos de comportamento:

Versão ingênua — incorreta e mais lenta que o sequencial: a ausência de sincronização gera resultados não-determinísticos, e a contenção de cache-line (falsa ou real) entre threads aumenta o tempo de execução em vez de reduzi-lo.

omp critical por ponto — correta mas 6,5× mais lenta que o sequencial: adquirir um lock global proporcional a N é o pior dos mundos — sincronização sem escalabilidade. Demonstra que sincronização excessiva é tão problemática quanto a ausência dela.

parallel+for+cláusulas e reduction — corretas e ~4× mais rápidas: ao acumular localmente e sincronizar apenas uma vez por thread (custo $O(\text{threads})$, não $O(N)$), obtém-se escalabilidade próxima ao ideal para 4 threads.

Versão	Correto?	Desempenho	Motivo
Sequencial	Sim	Baseline	—
Ingênua	Não	Pior	Race condition + contenção
critical por ponto	Sim	Pior ainda	Lock por iteração
parallel+for	Sim	~4× melhor	Critical apenas 1× por thread
reduction	Sim	~4× melhor	Forma idiomática do OpenMP

6. Conclusão

O experimento demonstrou que paralelizar código sem considerar o escopo das variáveis e a sincronização de acessos compartilhados produz resultados incorretos e desempenho inferior ao sequencial. A solução não está em adicionar mais locks, mas em minimizar o escopo crítico: acumular localmente e sincronizar apenas ao final.

A cláusula `reduction` do OpenMP encapsula esse padrão de forma idiomática, produzindo o código mais curto, mais legível e de maior desempenho. O uso de `default(none)` combinado com as demais cláusulas de escopo representa a prática recomendada para regiões paralelas complexas, pois força o programador a raciocinar explicitamente sobre cada variável compartilhada.

Speedup observado (reduction vs. sequencial): 0,94 s / 0,21 s $\approx$ 4,5 $\times$, próximo ao ideal para 4 threads.

7. Código Fonte

```
1  #define _POSIX_C_SOURCE 199309L
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <math.h>
5  #include <time.h>
6  #include <omp.h>
7
8  #define N      100000000L
9  #define SEED    12345u
10 #define PI_REF  3.14159265358979323846
11 #define BILLION 100000000.0
12 #define NTHREADS 4
13
14 static double elapsed(struct timespec a, struct timespec b) {
15     return (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / BILLION;
16 }
17
18 /* — 1. Versão sequencial ————— */
19 static double pi_sequential(void) {
20     long dentro = 0;
21     unsigned seed = SEED;
22     for (long i = 0; i < N; i++) {
23         double x = rand_r(&seed) / (double)RAND_MAX;
24         double y = rand_r(&seed) / (double)RAND_MAX;
25         if (x*x + y*y <= 1.0) dentro++;
26     }
27     return 4.0 * dentro / N;
28 }
29
30 /* — 2. Paralela ingênua: #pragma omp parallel for sem proteção —————
31     Problemas:
32     a) dentro++ não é atômico → race condition → resultado errado
33     b) seed compartilhada → race condition no próprio RNG */
34 static double pi_ingenua(void) {
35     long dentro = 0;
36     unsigned seed = SEED;
37     omp_set_num_threads(NTHREADS);
38     #pragma omp parallel for
39     for (long i = 0; i < N; i++) {
40         double x = rand_r(&seed) / (double)RAND_MAX;
41         double y = rand_r(&seed) / (double)RAND_MAX;
42         if (x*x + y*y <= 1.0)
43             dentro++; /* RACE CONDITION */
44     }
45     return 4.0 * dentro / N;
46 }
47
```

```

1 static double pi_critical(void) {
2     long dentro = 0;
3     omp_set_num_threads(NTHREADS);
4     #pragma omp parallel for
5     for (long i = 0; i < N; i++) {
6         unsigned seed = SEED ^ (unsigned)(i * 2654435761u); /* seed por iter */
7         double x = rand_r(&seed) / (double)RAND_MAX;
8         double y = rand_r(&seed) / (double)RAND_MAX;
9         if (x*x + y*y <= 1.0) {
10             #pragma omp critical
11             dentro++;
12         }
13     }
14     return 4.0 * dentro / N;
15 }

```

```

1 static double pi_clausulas(unsigned *seed_last_out) {
2     long dentro = 0;
3     long local; /* declarada no escopo externo para private() */
4     unsigned seed_master = SEED;
5     unsigned seed_last = 0;
6
7     omp_set_num_threads(NTHREADS);
8
9     #pragma omp parallel default(none) \
10     shared(dentro, seed_last) \
11     firstprivate(seed_master) \
12     private(local)
13     {
14         /* firstprivate: seed_master copiado do master; derivar seed única */
15         unsigned seed = seed_master ^ (unsigned)omp_get_thread_num();
16         local = 0;
17
18         #pragma omp for lastprivate(seed_last)
19         for (long i = 0; i < N; i++) {
20             double x = rand_r(&seed) / (double)RAND_MAX;
21             double y = rand_r(&seed) / (double)RAND_MAX;
22             if (x*x + y*y <= 1.0) local++;
23             /* lastprivate captura o seed da última iteração lógica (i == N-1) */
24             seed_last = seed;
25         }
26
27         #pragma omp critical
28         dentro += local;
29     }
30
31     if (seed_last_out) *seed_last_out = seed_last;
32     return 4.0 * dentro / N;
33 }

```

```

1 static double pi_reduction(void) {
2     long dentro = 0;
3     unsigned seed_master = SEED;
4
5     omp_set_num_threads(NTHREADS);
6
7     #pragma omp parallel for default(none) \
8         shared(seed_master) \
9         reduction(+:dentro)
10    for (long i = 0; i < N; i++) {
11        unsigned seed = seed_master ^ (unsigned)(omp_get_thread_num() * 2654435761u + (unsigned)i);
12        double x = rand_r(&seed) / (double)RAND_MAX;
13        double y = rand_r(&seed) / (double)RAND_MAX;
14        if (x*x + y*y <= 1.0) dentro++;
15    }
16    return 4.0 * dentro / N;
17 }

```

```

1 int main(void) {
2     struct timespec ini, fim;
3
4     printf("Estimativa de  $\pi$  – Monte Carlo (N = %ldM, %d threads)\n\n",
5           N / 1000000L, NTHREADS);
6     printf("%-32s %-12s %-10s %-8s\n",
7           "Versão", "π estimado", "Tempo(s)", "Correto?");
8     printf("%-32s %-12s %-10s %-8s\n",
9           "-----", "-----", "-----", "-----");
10
11    double pi, t;
12
13    #define MEDIR(label, expr, correto) \
14        clock_gettime(CLOCK_MONOTONIC, &ini); \
15        pi = (expr); \
16        clock_gettime(CLOCK_MONOTONIC, &fim); \
17        t = elapsed(ini, fim); \
18        printf("%-32s %-12.8f %-10.4f %-8s\n", \
19              (label), pi, t, (correto) ? "sim" : "NAO");
20
21    MEDIR("Sequencial", pi_sequencial(), 1)
22    MEDIR("Ingênua (race condition)", pi_ingenua(), fabs(pi - PI_REF) < 0.01)
23    MEDIR("Com omp critical", pi_critical(), 1)
24
25    unsigned seed_final;
26    clock_gettime(CLOCK_MONOTONIC, &ini);
27    pi = pi_clausulas(&seed_final);
28    clock_gettime(CLOCK_MONOTONIC, &fim);
29    t = elapsed(ini, fim);
30    printf("%-32s %-12.8f %-10.4f %-8s (seed_last=%u)\n",
31          "parallel+for+cláusulas", pi, t, "sim", seed_final);
32
33    MEDIR("Com reduction", pi_reduction(), 1)
34
35    printf("\nπ referência: %.8f\n", PI_REF);
36    return 0;
37 }

```